

Site de reservation

Backend Flask & PostgreSQL

Presentation orale - 20 minutes

Choix techniques - Methodologie - Securite - Communication front/back -
SEO - Limites - Questions/reponses

Sommaire

- 1. Choix techniques (Flask, Jinja, HTML/CSS, PostgreSQL/SQL)
- 2. Methodologie de developpement
- 3. Securite mise en place (avec emplacement precis dans le code)
- 4. Communication entre le backend et le frontend
- 5. SEO : ce qui a ete mis en place
- 6. Ce qu'on aurait pu ameliorer (et pourquoi on ne l'a pas fait)
- 7. Questions/reponses anticipees

1. Choix techniques

Pourquoi Flask ?

- Micro-framework : fournit le routage HTTP (= faire correspondre une URL visitée à une fonction Python précise qui doit y répondre), rien de plus - pas d'ORM imposé, pas de structure figée (contrairement à Django).
- Colle à l'objectif pédagogique : écrire le SQL à la main avec psycopg2, sans qu'un ORM (outil qui traduirait automatiquement des objets Python en requêtes SQL) masque le travail.
- Ecosystème suffisant : Flask-Login (sessions), Flask-Limiter (anti brute-force), Flask-WTF (CSRF).
- Contrainte d'hébergement : o2switch supporte Flask (WSGI) mais pas FastAPI (ASGI).
- Coherence avec une v1 du projet déjà développée en Flask (MongoDB) - réutilisation de la logique d'authentification.

Pourquoi Jinja2 ?

- Moteur de templates livré nativement avec Flask - aucune dépendance supplémentaire.
- Echappement automatique des variables affichées -> protection XSS par défaut.
- Permet de rendre le HTML côté serveur, avec les données déjà en base, sans passer par une API JSON séparée.

Détail sur l'échappement automatique (protection XSS) : ce comportement est automatique, activé par défaut par Flask pour tous les templates .html, sans rien coder soi-même. À chaque `{{ variable }}` affiché, Jinja convertit les caractères dangereux (`<` devient `<`, `>` devient `>`, `"` devient `"`, `&` devient `&`) avant de les insérer dans la page.

Exemple concret : dans `mes_destinations.html`, `{{ reserve.nom }}` `{{ reserve.prenom }}` affiche un nom/prénom saisi via un formulaire public (`reservation.html`). Si un utilisateur tapait `<script>alert('pirate')</script>` dans le champ nom, Jinja afficherait ce texte tel quel à l'écran au lieu de l'exécuter comme un vrai script dans le navigateur d'un autre visiteur.

Seule exception qui désactiverait cette protection : le filtre `| safe` (ex: `{{ variable | safe }}`), volontairement absent de tout le projet (vérifié : aucune occurrence sur les 13 templates) - donc l'échappement automatique s'applique partout, sans exception.

Organisation des fichiers Flask : templates/ et static/

- `templates/` : contient tous les fichiers HTML (Jinja). Flask les cherche automatiquement dans ce dossier dès qu'on appelle `render_template('nom.html')` - pas besoin de préciser le chemin complet.
- `static/` : contient tout ce qui est servi tel quel au navigateur, sans traitement côté serveur - CSS, JavaScript, images, logos. Accessible directement via l'URL `/static/...` (ex: `/static/css/styles.css`, `/static/assets/images/logo.webp`).
- Pourquoi cette séparation : Flask distingue le contenu dynamique (les templates, avec

variables Jinja injectees a chaque requete) du contenu fixe (static). Ca permet aussi au navigateur de mieux mettre en cache les fichiers static/ (ils ne changent pas a chaque requete, contrairement au HTML genere).

- Dans ce projet : les 13 templates (index.html, hotels.html, panier.html...) sont dans templates/, et static/ contient styles.css, script.js et les images (logo, visuels de la page d'accueil).

Pourquoi HTML/CSS classique (pas de framework JS) ?

- Le site est server-rendered : chaque page est generee cote Flask, pas de SPA (React/Vue) necessaire pour ce perimetre.
- Formulaire natifs (POST classique) + redirections -> simple, robuste, facile a securiser (CSRF, validation serveur).

Pourquoi PostgreSQL / SQL brut (pas d'ORM, pas de NoSQL) ?

- Objectif de l'examen : demontrer la maitrise du SQL - SGBD relationnel avec vraies contraintes (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL).
- Compare a la v1 (MongoDB, schema libre) : montre la difference entre un modele document flexible et un modele relationnel structure, avec l'integrite des donnees garantie par la base elle-meme.
- psycopg2 en SQL brut avec requetes parametrees (pas d'ORM comme SQLAlchemy, qui generait le SQL a notre place) -> chaque SELECT/INSERT/UPDATE/JOIN est ecrit et compris explicitement.

Precision : Flask et un ORM ne sont pas concurrents, ce sont deux couches differentes (Flask = routage HTTP, ORM = acces base de donnees) - on peut utiliser Flask AVEC un ORM (ex: Flask-SQLAlchemy). La vraie comparaison faite ici est SQL brut vs ORM, pas Flask vs ORM.

Avec un ORM, on ecrirait `Hotel.query.filter_by(slug=slug).first()` -> l'ORM genere le SQL a notre place, on ne le voit jamais. Avec psycopg2 (notre choix), on ecrit soi-meme `SELECT * FROM hotels WHERE slug = %s` -> on controle exactement la requete, ses jointures, ses conditions - rien n'est cache ni genere automatiquement.

2. Methodologie de developpement

"On a commence par le backend parce que..."

- La consigne de l'examen imposait de traiter le backend en premier, le front venant ensuite.
- Le modele de donnees est la fondation du projet : concevoir le schema SQL (tables, relations, contraintes) avant le HTML evite de devoir tout refaire si la structure change.
- Chaque route a ete testee isolément avec Postman (retour JSON) avant d'etre connectee a un vrai formulaire HTML - ca separe les bugs 'logique metier' des bugs 'affichage'.

Etapas suivies

- Mise en place PostgreSQL + connexion Python (Database/db.py)
- Conception du schema relationnel (tables + cles etrangeres)
- Migration des donnees depuis l'ancienne version MongoDB (scripts migrate_hotels.py, migrate_vols.py)
- Authentification (inscription / connexion / deconnexion / session)
- Routes metier (hotels, vols, recherche, contact)
- Parcours de reservation (panier, suppression, paiement simule)
- Connexion au frontend (templates Jinja existants, adaptation Mongo -> SQL)
- Sécurisation transverse (rate limiting, CSRF, SEO) appliquee en derniere passe sur toutes les routes

3. Securite mise en place

Chaque mesure est reliee a son emplacement reel dans app.py (numeros de ligne au moment de la redaction).

Le module Database/db.py : une connexion par requete

- `get_connection()` ouvre une connexion PostgreSQL via `psycopg2.connect(host=, dbname=, user=, password=)`, avec les identifiants lus depuis les variables d'environnement (`.env`) - jamais en dur dans le code.
- Chaque route ouvre sa propre connexion au debut, et la ferme systematiquement dans un bloc `finally` (`cursor.close() + conn.close()`), plutot qu'une connexion globale unique partagee par toute l'application.
- Pourquoi ce choix : au debut du projet, une connexion globale ouverte une seule fois au demarrage du serveur plantait des le 2e appel a une route (la premiere requete la fermait). Depuis, chaque requete = sa propre connexion isolee, ce qui evite ce bug et garantit qu'un utilisateur ne peut pas interferer avec la connexion d'un autre.
- Limite assumee (voir section 6) : pas de pool de connexions (`psycopg2.pool`) - acceptable pour le trafic d'un projet d'examen, a ameliorer en cas de forte charge en production.

Mode debug : jamais en production

- `app.run(debug=True)` reste correct en local, mais ne doit jamais etre code en dur pour la production : en mode debug, une erreur non geree affiche la stack trace complete (chemins de fichiers, extraits de code) au visiteur, et active un debogueur interactif exploitable par un attaquant.
- Nuance specifique au deploiement `o2switch` : avec `Passenger`, le fichier `passenger_wsgi.py` importe directement l'objet `app` (`from app import app as application`) sans jamais appeler `app.run()` - le bloc `if __name__ == '__main__'` n'est donc pas execute en production sur cet hebergement precis.
- Bonne pratique appliquee malgre tout : piloter le mode debug depuis une variable d'environnement plutot qu'une valeur figee ->
`app.run(debug=os.getenv('FLASK_DEBUG', 'False') == 'True')`, avec `FLASK_DEBUG=True` uniquement dans le `.env` local, absent du `.env` de production.

Comment on teste une route avec Postman

- 1. On identifie la route et sa methode (GET pour afficher un formulaire, POST pour le soumettre) dans `app.py`.
- 2. Pour une route protegee par CSRF : on fait d'abord un GET sur la page du formulaire pour recuperer le token cache dans le HTML (`{{ csrf_token() }}`), puis on l'inclut dans le corps du POST suivant (sinon la requete est rejete).
- 3. On envoie la requete avec le bon Content-Type (`form-urlencoded` pour un formulaire

classique, JSON pour une API) et les champs attendus par la route (verifies avec request.form['champ'] cote Flask).

- 4. On verifie trois choses : le code de statut HTTP retourne (200, 302, 400...), le contenu de la reponse, et l'effet reel en base de donnees (SELECT direct dans psql pour confirmer qu'une ligne a bien ete creee/modifiee/refusee).

Tests de securite reels effectues (preuves, pas juste relecture de code)

- Test 1 - Injection SQL sur /login : envoi du payload classique ' OR '1'='1' dans les champs email et mot de passe. Resultat : reponse HTTP 200 normale (pas de crash, pas d'erreur SQL exposee), connexion refusee - la requete parametree (%s) traite le payload comme une simple chaine de caracteres a comparer, jamais comme du SQL execute.
- Test 2 - Requete POST sans token CSRF sur /login : resultat exact obtenu -> HTTP 400 Bad Request, message "The CSRF token is missing." - preuve que Flask-WTF bloque bien toute soumission sans jeton valide.
- Test 3 - IDOR sur /delete_H : deux comptes reels crees (A et B). Avec le compte B connecte, tentative de suppression de la reservation appartenant a A via /delete_H/<id_de_A>. Resultat : la ligne de A reste intacte en base (0 ligne affectee par le DELETE, verifie par SELECT direct), alors que B peut normalement supprimer sa propre reservation. Preuve que la clause WHERE id = %s AND client_id = %s protege reellement, pas juste en theorie.

Mesure	Ou dans le code	Pourquoi
Injection SQL	Toutes les requetes : parametres %s + tuple. Ex. lignes 47, 104, 262, 303, 365, 422, 501-503	psycpg2 echappe automatiquement les valeurs ; impossible d'injecter du SQL via un champ de formulaire.
Mots de passe (bcrypt)	Hash a l'inscription : ligne 216. Verification au login : ligne 265	Hash + sel aleatoire a chaque inscription (bcrypt.gensalt()) - jamais de mot de passe stocke en clair.
Rate limiting (Flask-Limiter)	Lignes 156, 200, 247, 337, 398, 493, 515, 536 (@limiter.limit(...))	Limite les requetes par IP sur les routes sensibles (login, inscription, contact, reservation, paiement).
CSRF (Flask-WTF)	CSRFProtect(app) ligne 24, token cache dans les 9 formulaires POST du site	Empeche un site tiers de forcer une action au nom d'un utilisateur connecte a son insu.
Sessions securisees	secret_key ligne 20, duree ligne 22 (PERMANENT_SESSION_LIFETIME), session.permanent = True ligne 268	Cookie de session signe (infalsifiable sans la cle secrete), expiration apres 30 min d'inactivite.
Integrite en base (contraintes SQL)	UNIQUE sur l'email (compte_client), FOREIGN KEY sur reservations_hotel/reservations_vols	La base refuse elle-meme les doublons/incoherences, meme en cas de requetes concurrentes.
Protection IDOR	delete_H/delete_V lignes 502-503, 523-524 : WHERE id = %s AND client_id = %s	Un utilisateur ne peut supprimer/modifier que ses propres reservations, meme en changeant l'id dans l'URL. Teste en conditions reelles (2 comptes distincts).
Prix recalcule cote serveur	customers() lignes 365-368, customers_vols() lignes 422-425	Le tarif n'est jamais lu depuis un champ cache du formulaire - toujours recalcule depuis hotels/vols.

Gestion d'erreurs	try/except/finally systematique sur chaque route, debug=True uniquement en local	Aucune stack trace ni requete SQL exposee ; connexion toujours fermee meme en cas d'erreur.
Secrets hors du code	.env (mot de passe DB, SECRET_KEY) + .gitignore	Aucun secret n'est jamais commite sur GitHub.

4. Communication frontend <-> backend

- Rendu 100% cote serveur (SSR) : Flask execute la requete SQL, injecte les resultats dans un template Jinja via `render_template(...)`, renvoie du HTML deja pret - pas d'API JSON separee.
- Formulaires HTML classiques en POST (`application/x-www-form-urlencoded`), lus cote Flask avec `request.form['champ']`.
- Pattern Post/Redirect/Get : apres une action reussie (inscription, reservation, suppression), on fait un `redirect(url_for(...))` plutot que de renvoyer directement une page - evite la double soumission.
- Champs caches (`<input type="hidden">`) : transmettent un identifiant (`hotel_slug`, `vol_id`) du template vers la route suivante, sans jamais transmettre de donnee sensible modifiable.
- Cookie de session (Flask-Login) : envoye automatiquement par le navigateur a chaque requete, `current_user` disponible dans toutes les routes et templates.
- Token CSRF : genere cote serveur, injecte dans le template (`{{ csrf_token() }}`), renvoie dans le formulaire suivant - verifie par Flask-WTF avant de traiter la requete.

5. SEO : ce qui a ete mis en place

Balises meta uniques par page

- Constat de depart : quasiment toutes les pages partageaient le meme <title> ('Airlines reservation') et la meme <meta description> - probleme classique de contenu duplique, penalise par Google.
- Chaque page a reçu un <title> et une <meta description> uniques et descriptifs (ex : 'Nos Hotels | Airlines Reservation', 'Contactez-nous | Airlines Reservation').
- Pages de detail (hotel, vol) : titre et description generes dynamiquement avec Jinja a partir des donnees reelles ({{ hotel.nom }}, {{ vol.ville }}) - un titre different pour chaque hotel/vol, pas un texte fige.

Balise canonical

- Ajoutée sur toutes les pages : <link rel="canonical" href="{{ request.base_url }}">.
- Genere dynamiquement avec request.base_url plutot qu'une URL codee en dur (l'ancienne version pointait vers 127.0.0.1, fausse en production).

Meta robots sur les pages privees

- noindex, nofollow ajoute sur les pages qui n'ont aucun interet a apparaitre dans Google : creation de compte, connexion, panier, paiement, mes destinations, page de remerciement.
- Evite d'indexer des pages transactionnelles ou personnelles, qui degraderaient la qualite SEO percue du site.

Structure des titres H1/H2/H3

- Audit initial : 4 pages n'avaient aucun <h1> (dont la page d'accueil !), et plusieurs sautaient directement de h1 a h3 sans h2.
- Regle appliquee : un seul <h1> unique et descriptif par page, puis une hierarchie continue (h1 -> h2 -> h3) sans saut de niveau.
- Correction : ajout d'un h1 sur l'accueil, la liste hotels, la liste vols et les resultats de recherche ; passage des titres de cartes (nom d'hotel, ville) de h3 a h2.

Resultat verifie sur les 13 templates : exactement un h1 par page, hierarchie sans saut.

Resultats Google PageSpeed Insights (site en ligne, desktop)

- SEO : 100/100 - validation directe du travail fait (meta uniques, hierarchie de titres, canonical, HTML semantique).
- Bonnes pratiques : 96/100.
- Accessibilite : 72/100 - points a corriger releves par l'outil : champs de formulaire sans libelle associe, contraste insuffisant par endroits, absence de repere <main> dans le document.

- Performances : 57/100 - le point faible actuel. Largest Contentful Paint a 7,6s (trop lent) et Total Blocking Time a 410ms, alors que First Contentful Paint (0,7s), Speed Index (1,1s) et Cumulative Layout Shift (0.001) sont deja bons.
- Cause principale identifiee par l'outil : poids des images non optimisees (28 Mo d'economies estimees rien que sur l'affichage des images, page totale ~29 Mo) et ressources CSS bloquant le rendu (560ms d'economies estimees).

Piste d'amelioration concrete et rapide a mettre en oeuvre : compresser/redimensionner les images (formats web modernes comme WebP, tailles adaptees) et ajouter width/height explicites sur les balises pour eviter les decalages de mise en page.

Corrections appliquees depuis ce test : reperes <main> ajoutees sur les 13 pages, labels de formulaires associes a leurs champs (for/id), et le CSS Font Awesome (900ms de blocage) charge desormais en asynchrone (media="print" + bascule onload) plutot qu'en bloquant le rendu initial.

6. Ce qu'on aurait pu ameliorer

Choix assume : rester sur des mecanismes simples et solides plutot que d'ajouter de la complexite pour l'examen. Pistes identifiees pour une vraie mise en production :

- JWT / API stateless - utile pour une app mobile ou un frontend JS separe ; pas necessaire ici car le site est mono-domaine et server-rendered (le cookie de session suffit).
- Reinitialisation de mot de passe par email et verification d'email a l'inscription.
- Double authentication (2FA) pour les comptes clients.
- Pool de connexions PostgreSQL (psycopg2.pool) au lieu d'ouvrir/fermer une connexion a chaque requete.
- Tests automatises (pytest + client de test Flask) - actuellement teste manuellement (Postman + navigateur).
- Decoupage en Blueprints Flask - app.py est un seul fichier, un vrai projet de prod separerait auth/hotels/vols/paiement en modules.
- HTTPS/HSTS + en-tetes de securite (Content-Security-Policy, etc.) - non pertinent en developpement local.
- Vraie passerelle de paiement (Stripe, etc.) - le formulaire de carte bancaire actuel est simule.
- Stockage des photos sur un service dedie (S3, Cloudinary) au lieu de simples URLs en base.
- Optimisation des images (compression, format WebP, dimensions adaptees, attributs width/height) - identifie concretement par le test PageSpeed Insights (score Performances 57/100, ~28 Mo d'economies estimees).

7. Questions/reponses anticipees

Q. Pourquoi ne pas avoir utilise Django ou un ORM comme SQLAlchemy ?

R. Un ORM (outil qui traduit automatiquement des objets Python en requetes SQL) genere le SQL a notre place et masquerait justement le travail que l'examen demande de demontrer : ecrire et maitriser le SQL soi-meme.

Q. Comment vous protegez-vous des injections SQL ?

R. Toutes les requetes utilisent des parametres %s avec psycopg2, jamais de concatenation de chaines ni de f-string avec des donnees utilisateur.

Q. Comment sont stockes les mots de passe ?

R. Hashes avec bcrypt (algorithme concu pour etre lent, donc resistant au brute-force), avec un sel aleatoire different a chaque inscription. Jamais stockes ni affiches en clair.

Q. Qu'est-ce que le CSRF et comment le gerez-vous ?

R. Une attaque ou un site tiers force le navigateur d'un utilisateur connecte a envoyer une requete a son insu. On s'en protege avec un token unique genere par Flask-WTF, injecte dans chaque formulaire.

Q. Comment gerez-vous les sessions utilisateur ?

R. Avec Flask-Login : un cookie signe cryptographiquement (grace a la SECRET_KEY) stocke l'identite de l'utilisateur, avec une expiration automatique apres 30 minutes d'inactivite.

Q. Qu'est-ce qu'une faille IDOR et comment vous en protegez-vous ?

R. Acceder aux donnees d'un autre utilisateur en changeant un id dans l'URL. On s'en protege en verifiant systematiquement que la ressource appartient bien au client_id de l'utilisateur connecte - teste en conditions reelles avec 2 comptes distincts.

Q. Comment le prix d'une reservation est-il securise ?

R. Il n'est jamais lu depuis le formulaire (un champ cache serait modifiable) : il est recalcule cote serveur a partir du tarif reel stocke en base.

Q. Pourquoi PostgreSQL plutot que MySQL ou SQLite ?

R. PostgreSQL est un SGBD relationnel robuste, avec des contraintes riches (CHECK, FOREIGN KEY) et proche d'un vrai environnement de production.

Q. Pourquoi une connexion a la base par requete plutot qu'une connexion globale ?

R. Une connexion globale ouverte une seule fois au demarrage a cause un bug tot dans le projet (elle se fermait des la premiere requete et cassait toutes les suivantes). Une connexion par requete, fermee dans un bloc finally, isole chaque utilisateur et evite ce probleme - la limite etant l'absence de pool de connexions pour l'instant.

Q. Quelles sont les limites que vous reconnaissez a ce projet ?

R. Pas de vraie passerelle de paiement, pas de tests automatises, une connexion base ouverte par requete plutot qu'un pool, un seul fichier app.py non decoupe en modules.

Q. Avez-vous teste votre application ?

R. Oui, manuellement : chaque route testee isolément avec Postman avant integration au frontend, puis testee en conditions reelles dans le navigateur pour tout le parcours.

Q. Comment avez-vous travaille le referencement (SEO) ?

R. Titre et meta description uniques sur chaque page (fini le contenu duplique), balise canonical dynamique, meta robots noindex sur les pages privees, et une hierarchie de titres h1/h2/h3 propre sur les 13 templates (un seul h1 par page).

Q. Avez-vous vraiment teste les failles de securite, ou juste relu le code ?

R. Teste pour de vrai avec des requetes reelles (via curl/Postman) : injection SQL sur le login (payload ' OR '1'='1', refuse sans crash), requete sans token CSRF (400 Bad Request, message explicite), et IDOR avec deux comptes reels (impossible pour un compte de supprimer la reservation d'un autre).

Q. Avez-vous teste le score avec Google PageSpeed Insights ?

R. Oui, sur le site deploye : SEO 100/100, Bonnes pratiques 96/100, Accessibilite 72/100, Performances 57/100. Le point faible est le poids des images non optimisees (LCP a 7,6s) - une piste d'amelioration identifiee mais pas encore corrige, faute de temps avant l'examen.

Document de preparation a l'oral - Site de reservation (Flask + PostgreSQL).